

auth-service

Qué hace el servicio y qué hay dentro de las carpetas security, service, config y model.

Este documento explica, en lenguaje simple, el servicio de autenticación de la tienda: cuál es su responsabilidad, qué guarda, y qué hace cada una de sus carpetas. Al final hay un recorrido paso a paso de un login completo, para ver cómo encajan todas las piezas.

¿Qué es auth-service?

Es el **portero** de la aplicación: el único servicio que sabe de contraseñas. Su trabajo es comprobar que sos quien decís ser y darte un **pase** (un token JWT firmado) para que puedas moverte por el resto de la app sin volver a identificarte.

La idea clave es que está **deliberadamente incompleto**: sabe probar tu identidad, pero no sabe quién sos.

Lo que SÍ maneja

- El **hash** de las contraseñas (BCrypt). Nunca guarda texto plano.
- **Firmar los JWT**: es el único que tiene la clave privada RSA.
- **Refresh tokens**, para renovar la sesión sin volver a loguearte.
- **2FA / TOTP** (Google Authenticator).
- **Bloqueo por intentos fallidos**.
- El flujo de **login con Google**.

Lo que NO maneja

- Tu **nombre, email o rol** (ADMIN / VENDEDOR / COMPRADOR).
- Si tu cuenta está **habilitada** o no.

Todo eso vive en el servicio de negocio, y auth-service **se lo pregunta por HTTP durante el login**. Por eso tiene una carpeta ``client/``.

model/ — las tablas de la base

Son las "cajas" donde se guarda la información. Cada clase equivale a una tabla de la base de datos. Acá sólo hay **dos**, y eso ya dice mucho sobre lo poco que este servicio necesita saber.

Credential.java — la parte secreta del usuario

Es lo que quedó del usuario después de sacarle todo lo que no es autenticación. Sus campos:

- ``username`` — la **clave natural** que comparte con el perfil del usuario.
- ``passwordHash`` — la contraseña encriptada con BCrypt.
- ``email`` — una copia, necesaria para casar la cuenta en el login con Google.
- ``intentosFallidos`` + ``bloqueadoHasta`` — los contadores del bloqueo automático.
- ``totpSecret`` + ``totpHabilitado`` — el segundo factor.

Fíjate que **no hay rol ni flag de habilitado**: eso es a propósito.

RefreshToken.java — el "pase largo"

Un texto opaco (un UUID) que dura 7 días. Se guarda en la base por una razón concreta: **para poder revocarlo**. Un JWT suelto no se puede anular antes de que expire; un registro en la base sí.

- ``token``, ``username``, ``expiraEn``, ``revocado``
- ``esValido()`` — devuelve true sólo si **no** está revocado **y** todavía no venció.

security/ — las herramientas de seguridad

Acá vive la maquinaria de Spring Security: quién puede entrar a cada URL, cómo se compara una contraseña, y los mecanismos extra (2FA, bloqueo, Google).

SecurityConfig.java

El **plano de las puertas**. Define qué URL es pública y cuál necesita token, y arma las piezas base (el codificador BCrypt y el ``AuthenticationManager``).

- ``/api/auth/login``, ``/refresh``, ``/logout`` → **públicas**.
- ``/internal/`` → **sólo para otros servicios (header ``X-Internal-Token``)**.
- El resto → hace falta un JWT válido.

Detalle: usa sesión ``IF_REQUIRED`` en vez de ``STATELESS`` porque el login con Google necesita sesión para su ida y vuelta de redirects.

CredentialUserDetails.java

Un **traductor**. Spring Security espera un objeto ``UserDetails``; esta clase envuelve a ``Credential`` para que Spring pueda comparar el hash y aplicar el bloqueo.

Detalle importante

``isEnabled()`` siempre devuelve **true**. No es un descuido: el flag "habilitado" no vive en este servicio, así que lo chequea ``AuthService`` más tarde, contra la respuesta del servicio de negocio.

CredentialUserDetailsService.java

Tres líneas: busca la credencial por `username` en la base. Es el puente entre Spring Security y el repositorio.

LoginAttemptListener.java

El **guardia de los intentos**. No lo llama nadie directamente: escucha los eventos que publica el `AuthenticationManager`.

- **5 fallos seguidos** → cuenta bloqueada **15 minutos**.
- Un login exitoso **resetea** el contador.

TotpService.java

El **segundo factor**. Implementa TOTP (RFC 6238) a mano: el mismo algoritmo que usan Google Authenticator o Authy.

- `generarSecreto()` — crea la clave secreta del usuario.
- `generarUri()` — arma el texto del **código QR** que escanea la app.
- `validar()` — comprueba el código de 6 dígitos, con tolerancia de ± 30 segundos (por si el reloj del teléfono está corrido).

OAuth2ClientConfig.java

Registra a **Google** como proveedor de login. Está escrito a mano para que el login con Google sea **opcional de verdad**: si no configurás las credenciales, la app arranca igual y el resto sigue funcionando.

OAuth2LoginSuccessHandler.java

Qué hacer cuando Google dice **"sí, es él"**:

1. Pide el perfil al servicio de negocio, que lo crea si no existe (con rol **COMPRADOR**, el más restrictivo).
2. Se asegura de que exista una credencial local (con password random inutilizable).
3. Emite el access token y el refresh token.
4. Redirige al frontend con los dos tokens en la URL.

service/ — el cerebro (la lógica de negocio)

Si ``security/`` son las herramientas, ``service/`` es **quién decide en qué orden se usan**. Acá está la lógica propia del login, independiente de cómo entre la petición.

AuthService.java

El **director de orquesta**. Es la clase más importante del servicio. El orden de los pasos no es casual: cada uno puede cortar el login.

1. Valida usuario + contraseña contra la base local.
2. Le pregunta al servicio de negocio el **rol** y si está **habilitado**.
3. Si la cuenta tiene 2FA, exige y valida el código de 6 dígitos.
4. Recién ahí emite el access token + el refresh token.

También maneja ``refresh()`` (renovar la sesión sin re-loguearse) y ``logout()`` (revocar el refresh token).

CredentialService.java

El **alta, baja y modificación** de credenciales. Lo interesante es **quién lo llama**: no el usuario final, sino el servicio de negocio a través de la API interna, cuando el ADMIN crea o edita un usuario desde el panel. El perfil se guarda allá, la credencial acá.

RefreshTokenService.java

Crear, **rotar** y revocar refresh tokens. La rotación es la parte interesante: cada vez que usás un refresh token, ese token se revoca y se te entrega uno nuevo.

¿Por qué rotar?

Si alguien te robó un refresh token e intenta usarlo después que vos, ya no sirve. El intento fallido **es la señal** de que hubo una filtración.

AuthExceptions.java

Los tres errores típicos del login, que después se traducen a JSON con el código HTTP correcto:

- ``DosFactoresRequerido`` — falta el código 2FA, o está mal.
- ``CuentaDeshabilitada`` — el ADMIN deshabilitó la cuenta.
- ``NoAutorizado`` — contraseña incorrecta, cuenta bloqueada o sesión expirada.

config/ — el arranque y los ajustes

Cosas que se hacen **una vez, al encender el servicio**. No tienen lógica de negocio.

DataInitializer.java

Carga los usuarios de prueba al arrancar, para que la demo tenga con qué trabajar:

- ``admin`` / ``admin123``
- ``vendedor1``, ``vendedor2`` / ``vendedor123``
- ``comprador1`` ... ``comprador5`` / ``comprador123``

Es **idempotente**: si el usuario ya existe, no lo toca. Podés reiniciar sin miedo a pisar datos.

RestClientConfig.java

Pone **timeouts cortos** (2 segundos para conectar, 3 para leer) a las llamadas HTTP hacia otros servicios. La idea: si el otro servicio no contesta rápido, es mejor fallar limpio que dejar cada login colgado.

Cómo encaja todo: un login paso a paso

El recorrido completo, desde que apretás "Entrar" hasta que tenés el token:

1. El navegador manda usuario + contraseña a `POST /api/auth/login`.
2. `AuthController` (carpeta `web`) recibe la petición y llama a **AuthService**.
3. **AuthService** delega en el `AuthenticationManager`, que usa `CredentialUserDetailsService` para buscar la fila en la tabla `credenciales`.
4. Se compara la contraseña con el hash usando **BCrypt**.
5. Si falla → **LoginAttemptListener** suma un intento. A los 5, bloquea 15 minutos.
6. Si la contraseña está bien → `UsuariosClient` pregunta al servicio de negocio el **rol** y si está **habilitado**.
7. Si la cuenta tiene 2FA → **TotpService** valida el código de 6 dígitos.
8. `JwtService` firma el **access token** (15 min) y **RefreshTokenService** crea el **refresh token** (7 días).
9. El navegador guarda los dos y manda el access token en cada petición siguiente.

Las otras carpetas, en una línea

- `web` — los **controladores**: reciben el HTTP y traducen a llamadas del service.
- `repository` — el **acceso a la base** (Spring Data JPA / Hibernate).
- `client` — las **llamadas salientes** al servicio de negocio, con reintentos y circuit breaker.

Nota sobre la unificación reciente

Al unificar `catalogo-service` y `usuarios-service` en **negocio-service**, los endpoints `/internal/usuarios` que **auth-service** consume pasaron a ser servidos por **negocio-service** (mismo puerto 8082). No hay que cambiar código de auth-service: sólo apuntar la propiedad `app.services.usuarios-url` al servicio nuevo.